Trade Order Service: Hands-On Project Scope

Prepared for Vaishali Patil. Each section maps to one resume bullet, with deliverables and a "done when" check. Only claim on the resume what is built and verified.

Architecture at a glance

Dashboard (browser) → Auth.Service (login, tokens) → Orders API (`/v1`) → PostgreSQL (orders, outbox, ingestion jobs, audit log) → RabbitMQ → Order Worker. The MCP server and the Python data-quality CLI sit beside the API. Consul handles service discovery, and OpenTelemetry feeds Grafana.

Sample data: orders with symbol, quantity, side, account ID, price and timestamp, supplied as CSV and JSON files. The S3 bucket holds these files and the data-quality reports.

Build order and priority

1. **Must finish before the Schwab meeting:** Orders API, RabbitMQ worker, Dockerfiles, Docker Compose, README.
2. **Next:** OAuth2, rate limiting, ingestion framework, MCP server, data-quality library.
3. **Then:** dashboard, spec-first CI check, telemetry and runbook.
4. **Last:** Kubernetes and Helm, Consul, AWS deployment, PowerShell.

If time runs short, cut from the bottom of this list. A working Compose version with a documented Kubernetes and AWS plan beats a half-finished everything.

1. Versioned, OpenAPI-first REST API with OAuth2, rate limiting and a dashboard

1.1 Orders API (ASP.NET Core)

- `POST /v1/orders` with an `Idempotency-Key` header so duplicate submits are safe.
- `GET /v1/orders/{id}` and `GET /v1/orders` (list, with filters and paging).
- `GET /v1/data-quality/reports` for the dashboard and the MCP tool.
- Health endpoints: `/health/live` and `/health/ready`.

- *Done when:* submitting the same idempotency key twice creates one order.

1.2 Spec-first OpenAPI and versioning

- Write `openapi.yaml` first, then implement against it.
- CI validates the spec and fails if the implementation differs.
- Version in the URL (`/v1`). Document the rule for `/v2` : only breaking changes get a new version, and `/v1` stays available for a stated deprecation window.
- *Done when:* a deliberate mismatch fails the CI build.

1.3 OAuth2 (bare minimum identity service)

A single `Auth.Service` project using Open.IdentityServer with its ASP.NET Identity and EF packages, one SQL database, no admin UI. Check the package's maintenance status before depending on it. Keycloak is the fallback.

- **Scopes:** `orders.read` and `orders.write` . GET requires read, POST requires write.
- **Client credentials:** `mcp-server` (read only) and `order-worker` (read and write), seeded at startup.
- **Authorization code with PKCE:** one public client, `dashboard` , no secret, one redirect URI, with a basic Razor login page.
- **ASP.NET Identity users in SQL:** two seeded users (read only; read and write), lockout after 5 failed attempts.
- The Orders API validates tokens through OIDC discovery.
- *Done when:* no token gives 401, wrong scope gives 403, the MCP client cannot POST, and the sixth bad password attempt is locked out.

1.4 Rate limiting

- ASP.NET Core's built-in rate limiter, partitioned per client ID.
- Return 429 with a `Retry-After` header.
- *Done when:* a quick request loop triggers 429 and the README shows it.

1.5 Dashboard

- A small page (plain HTML and JS) that logs in through authorization code with PKCE.
- Shows recent orders, order status and data-quality failures from the API.
- *Done when:* it runs in Docker Compose beside the API.

1.6 Messaging and worker

- The API writes the order and an outbox row in one transaction. A publisher sends `OrderPlaced` to RabbitMQ.
- The worker validates and "fills" the order, then publishes `OrderFilled`. It is idempotent.
- Failed messages go to a dead-letter queue with a reason code.
- Polly provides retries with backoff and a circuit breaker on one downstream call.
- *Done when:* a message processed twice produces one fill, and a bad message lands in the dead-letter queue.

2. Reusable ingestion framework with pluggable adapters and job tracking

- Define an `ISourceAdapter` interface and a shared base class that templates the common steps (read, map, validate). Provide CSV and JSON adapters, registered through DI so new adapters need no core changes.
- Load idempotently, keyed on a hash of each record, into SQL.
- `IngestionJobs` table: job ID, source, status, records read, loaded and rejected, start and finish times.
- Run the Python data-quality checks as a pipeline step.
- *Done when:* loading the same file twice gives the same row count and two job records.

3. Python data-quality library and MCP server

3.1 Data-quality library (Python)

Three checks per dataset, run on order data and event payloads:

1. **Schema and required fields:** symbol, quantity, side and account ID present and correctly typed.
2. **Validity rules:** quantity above 0, side is BUY or SELL, price in a sane range.
3. **Duplicates and freshness:** no repeated idempotency keys, no stale or future timestamps.

Runs as a containerized CLI step called by the ingestion job. Results go to a table and the S3 bucket. Failed records go to the dead-letter queue with a reason code.

3.2 MCP server (4 tools)

- `get_order_status(order_id)` : read only.
- `list_recent_orders(filter)` : read only.
- `get_data_quality_report(dataset)` : read only.
- `replay_dead_letter_message(id)` : a write action, disabled by default.

3.3 Guardrails

- **Read-only default:** write tools stay off unless a config flag is set.
- **Approval gate:** the replay tool returns a pending-approval token, and a human confirms before it runs.
- **Audit log:** every tool call records the caller (client ID), arguments, result and timestamp in an append-only table.
- *Done when:* the replay tool refuses without the flag, will not run without approval, and every call appears in the audit table.

4. Containers, Kubernetes with Helm, AWS, GitHub Actions and PowerShell

4.1 Docker

- Multi-stage Dockerfiles for the API, worker and Auth.Service (SDK build stage, small `aspnet` runtime stage), non-root user, `.dockerignore`, `HEALTHCHECK`.
- GitHub Actions builds and tags images with the commit SHA.
- Docker Compose runs the whole stack locally.

4.2 Kubernetes and Helm (kind or Docker Desktop)

- Deployments and Services for the API, worker and Auth.Service, plus RabbitMQ and Postgres.
- ConfigMaps and Secrets for connection strings, no hardcoded values.
- Liveness and readiness probes tied to the health endpoints, resource requests and limits, 2 API replicas.
- An Ingress, or YARP as the gateway, as the entry point.
- Packaged as a Helm chart with `values-dev.yaml`.

4.3 Consul

- Install with the official Consul Helm chart.
- Services register on startup and deregister on shutdown.
- The API finds the worker or another downstream service by name through Consul, not a hardcoded URL.
- Consul health checks call `/health/live`. Add one shared value in Consul KV.
- *Done when:* stopping a service removes it from Consul and the API stops routing to it.

4.4 AWS deployment

- **S3:** a bucket for sample input files and data-quality reports.
- **EC2:** run the Compose stack on one instance.
- **GitHub Actions:** extend the workflow to deploy as well as build and test.
- *Done when:* the service responds on the EC2 public address.

4.5 PowerShell

- One script for build, push and deploy.
- One script for health check and restart.

5. Telemetry, runbook and tested recovery

- **Metrics:** OpenTelemetry counters and timers (orders per minute, API latency, queue depth, failed messages), exported to Prometheus.
- **Logs:** Serilog structured logs with a correlation ID flowing through API, queue and worker.
- **Traces:** OpenTelemetry spans across API, queue and worker.
- **Grafana:** a dashboard with 3 or 4 panels: latency, queue depth, error rate.
- **`RUNBOOK.md`:** 3 scenarios, each with symptoms, how to confirm, how to fix, and how to verify recovery.
- **Tested scenarios, each timed from failure to healthy:**
  1. Kill the worker pod and confirm Kubernetes restarts it with no lost messages.
  2. Stop RabbitMQ and confirm the outbox lets the API recover without losing orders.
  3. Send bad messages and confirm they land in the dead-letter queue.

- Record the measured recovery times in the README. Use the "3 minutes" claim only if the measurement supports it.

README requirements

- Architecture diagram and a quick-start (`docker compose up`).
- Rate-limit demo, scenario timings and the audit-log example.
- **Design decisions and what I would change at scale:**
  - Kafka versus RabbitMQ.
  - Why exactly-once is a myth, and how idempotent consumers deal with at-least-once delivery.
  - The outbox pattern.
  - Not built, would add at scale: delegated (on-behalf-of) tokens, 2FA, external identity federation, per-record entitlement checks, Redis sessions, per-tool authorization for MCP.

Resume check

Resume bullet	Section
Versioned, OpenAPI-first API with MCP, OAuth2, rate limiting, dashboard	1 and 3.2
Reusable ingestion framework, pluggable adapters, idempotent loads, job tracking	2
Python data-quality library (3 checks), MCP server (4 tools, read-only, approvals, audit)	3
Docker, Kubernetes with Helm, S3 and EC2, GitHub Actions, PowerShell	4
Telemetry, runbook, tested scenarios, recovery time	5